

Programação Concorrente - Trabalho final

Ricardo Martins Fortes, 190044179
UnB - Departamento de Ciência da Computação
CIC0202 - 2026.01

Link para repositório no GitHub: <https://github.com/ricardomf01/Programa-o-Concorrente---Projeto-Final>

I. INTRODUÇÃO

N O início da computação, as máquinas executavam apenas uma tarefa por vez. Com a evolução do hardware, surgiu a necessidade de aproveitar melhor a capacidade dos processadores, dando origem aos processos — programas em execução com seu próprio espaço de memória isolado [1]. Para reduzir o custo de alternar entre processos independentes e pesados, as threads surgiram como tarefas menores, compartilhando a mesma memória de um processo principal [2]. Isso permitiu consolidar a programação concorrente, onde múltiplas partes de um mesmo software rodam ao mesmo tempo para otimizar o desempenho.

Naturalmente, surgiram na literatura problemas clássicos de sincronização. O problema do produtor-consumidor lida com o equilíbrio entre quem gera dados e quem os consome dentro de um espaço limitado, garantindo que o produtor pare quando o armazenamento estiver cheio e o consumidor aguarde quando estiver vazio. Já o problema dos leitores e escritores simula o acesso a dados comuns: várias threads podem ler o dado juntas, mas quem precisa modificar o recurso exige acesso totalmente exclusivo.

O conceito de controle de capacidade representa o gerenciamento prático de quantos indivíduos podem utilizar um recurso físico limitado ao mesmo tempo. O gerenciamento inadequado dessa capacidade pode levar a falhas graves de sincronização, fazendo com que threads fiquem bloqueadas indefinidamente, aguardando por uma vaga que nunca é liberada pelo sistema [3].

A. Mutexes e conceitos correlatos

Quando várias threads compartilham o mesmo espaço de memória e tentam modificar a mesma variável juntas, ocorre a condição de corrida, fenômeno onde o resultado final se torna imprevisível e depende da velocidade e da ordem em que o sistema operacional em que o processador executa cada tarefa [?]. Para evitar esse problema, aplica-se o conceito de exclusão mútua através do mutex, que funciona como uma trava de segurança única. A thread que precisa acessar uma região crítica do código pega essa chave, "tranca a porta" para realizar o seu trabalho e, ao terminar, devolve a chave, fazendo com que qualquer outra thread interessada nessa região espere pacientemente a sua vez.

Relacionado com os mutexes, as variáveis de condição são uma ferramenta que permitem que uma thread interrompa seu próprio trabalho e durma voluntariamente, caso uma regra

de negócio específica não tenha sido atendida. Isso impede o desperdício de processamento, evitando que a thread fique checando uma variável sem parar em um laço de repetição ruidoso, oferecendo uma estratégia eficiente para acordar ou adormecer tarefas sinalizadoras.

B. Semáforos

Idealizados originalmente pelo cientista da computação Edsger Dijkstra [4], os semáforos são ferramentas de sincronização flexíveis que funcionam como contadores de permissões protegidos. Eles controlam o fluxo de execução por meio de duas ações simples e indivisíveis: uma que diminui o contador quando uma linha de execução solicita um recurso (operação de espera) e outra que aumenta o contador quando o recurso é liberado (operação de sinalização). Se o contador chegar a zero, as próximas threads que pedirem acesso serão bloqueadas e colocadas em uma fila.

Existem dois tipos principais de semáforos: os binários, que possuem apenas os valores zero e um e funcionam de forma idêntica a um mutex, garantindo exclusão mútua simples, e os de contagem, que podem ser iniciados com qualquer valor positivo para gerenciar uma quantidade exata de vagas em recursos físicos, como assentos ou vagas disponíveis. A grande vantagem do semáforo sobre o mutex é que uma thread pode sinalizar e liberar um semáforo que foi travado por outra, tornando-o a ferramenta ideal para coordenar a comunicação direta entre produtores e consumidores e gerenciar prioridades no sistema.

C. Interruptores e Catracas

Para coordenar a entrada e a saída de grupos de tarefas que compartilham interesses comuns, surgiram padrões lógicos elegantes, como o interruptor e a catraca. O padrão interruptor (*lightswitch* em inglês) funciona de maneira bastante intuitiva: a primeira thread de um grupo específico que acessa o recurso compartilhado "acende a luz" e tranca a porta de entrada para os grupos rivais. Enquanto a luz estiver acesa, outros membros do mesmo grupo podem entrar livremente na área sem precisar da chave principal. Apenas a última thread a concluir sua tarefa e sair assume a responsabilidade de "apagar a luz" e destravar o recurso. Contudo, se esse fluxo de membros do mesmo grupo for contínuo, os rivais podem acabar esperando para sempre. É para evitar essa injustiça que se utiliza a catraca (*turnstile*), um mecanismo que atua como uma fila de triagem. Quando uma thread adversária chega e tenta passar pela catraca, ela cria uma barreira invisível que impede temporariamente a entrada de novos membros do grupo dominante. Dessa forma, a área de uso eventualmente se esvazia de forma natural, e o direito

de acesso é repassado, garantindo um revezamento justo para todo o sistema.

II. PROBLEMA PROPOSTO: *O Problema das vilas de Canibais Guerreiros*

Neste cenário, as duas vilas de canibais guerreiros alimentam uma rivalidade feroz entre si, porém, existe apenas um cozinheiro capaz de preparar a carne que sacia estes guerreiros, e uma única mesa de refeições próxima ao caldeirão, que são compartilhados.

Portanto, é necessário que os guerreiros de cada vila não se alimentem ao mesmo tempo, para evitar tensões e uma eventual guerra. Cada vila também possui um cacique, que devido ao seu cargo, não come juntamente dos demais guerreiros. Ambos os caciques, entretanto, possuem habilidades diplomáticas para encontrar um ao outro na mesa, caso necessário.

A. Descrição Geral do Sistema

O sistema modela um cenário clássico de concorrência envolvendo produtores e consumidores, com múltiplas classes de usuários e regras rígidas de exclusão mútua. O cenário descreve três facções distintas que precisam compartilhar recursos comuns de alimentação: um caldeirão, que atua como recurso consumível, e uma mesa, que atua como recurso físico com espaço limitado. Para a reposição dos alimentos, o sistema conta com o suporte de um único cozinheiro neutro.

B. Entidades e Recursos Compartilhados

As entidades ativas que compõem o sistema são categorizadas em produtores e consumidores. A entidade produtora é singular e representada pelo cozinheiro, responsável exclusivo pelo preparo das refeições. As entidades consumidoras são divididas estritamente em três grupos ou facções independentes: os guerreiros da Vila 1, os guerreiros da Vila 2 e um terceiro grupo formado pelos dois caciques.

Os recursos passivos disputados pelas entidades são:

- **O Caldeirão:** Um repositório de refeições com capacidade máxima predefinida. Ele inicia a rotina completamente cheio e tem suas porções consumidas de forma individual por cada canibal.
- **A Mesa:** Representa o espaço físico do ambiente e determina o nível de concorrência simultânea, possuindo um número fixo e limitado de lugares (cadeiras).

C. Regras de Acesso e Exclusão Mútua

Para garantir consistência e respeito à lógica do cenário, as seguintes restrições devem ser aplicadas:

- **Restrição de Capacidade:** O número de indivíduos se alimentando ao mesmo tempo nunca pode ultrapassar o limite de cadeiras disponíveis na mesa. Logicamente, é proibido o consumo de porções de um caldeirão vazio.
- **Exclusão Mútua Categórica:** A mesa é um ambiente estritamente exclusivo para um grupo por vez. Se o espaço estiver sendo ocupado por membros de uma determinada facção (seja a Vila 1, a Vila 2 ou o grupo dos caciques), a entrada de qualquer indivíduo pertencente às

outras duas facções é terminantemente bloqueada. Neste modelo, os caciques não possuem privilégios hierárquicos para furar filas, operando sob as mesmas regras de disputa e restrição que as vilas de guerreiros.

D. O Comportamento do Produtor

A reposição dos recursos segue uma lógica puramente reativa. O cozinheiro permanece em estado de dormência até ser explicitamente sinalizado por um consumidor. Essa notificação possui uma condição: ela acontece apenas quando o caldeirão atinge zero porções.

Uma vez acordado, o cozinheiro assume o controle exclusivo do caldeirão e prepara um lote completo de alimento, preenchendo-o até a sua capacidade máxima. Durante esse período de preparo, todo e qualquer consumidor que estiver na mesa e encontrar a panela vazia deve aguardar o fim do preparo. Concluída a tarefa e com o recurso devidamente reabastecido, o cozinheiro retorna ao seu estado de repouso.

E. Requisitos de Justiça e Vivacidade

O modelo exige a garantia de que o sistema opere de forma contínua e livre de falhas críticas de concorrência, devendo suportar duas propriedades fundamentais:

- **Ausência de Impasses (Deadlock):** O fluxo de controle deve garantir que o sistema nunca atinja um estado de bloqueio eterno, cenário em que as entidades ficariam presas para sempre aguardando recursos que nunca seriam liberados.
- **Ausência de Inanição (Starvation):** Nenhuma facção pode sequestrar o direito de uso da mesa indefinidamente. O sistema impõe um mecanismo de triagem perfeitamente justo: se uma facção está ocupando a mesa de forma contínua e um membro de uma facção rival chega à fila de espera, o acesso de novos membros do grupo dominante deve ser temporariamente interrompido. Isso assegura que a área se esvazie em tempo finito e o turno seja repassado com justiça para os demais grupos.

III. IMPLEMENTAÇÃO E ANÁLISE DO PROBLEMA

A. Estrutura Geral: Variáveis Globais e Função Principal

A arquitetura do projeto foi construída sobre um conjunto de variáveis globais que representam os recursos físicos do problema e os mecanismos de controle de acesso. Foi utilizado semáforos para coordenar o fluxo de tarefas, como a catraca de entrada, as vagas na mesa e a comunicação com o cozinheiro. Em paralelo, os mutexes (travas de exclusão mútua) foram aplicados para proteger o incremento e decremento de contadores sensíveis, como o nível de porções no caldeirão e as cadeiras ocupadas no momento.

A função principal (main) realiza o papel de coordenadora do programa. Sua responsabilidade inicial é inicializar os semáforos com suas capacidades corretas — por exemplo, a mesa começa com vagas equivalentes à sua capacidade máxima, enquanto o cozinheiro começa com zero chamados, ou seja, dormindo. Após essa preparação, a função main cria as threads de forma independente: um cozinheiro, dois caciques

e os grupos de guerreiros de cada vila. Por fim, o sistema entra em estado de espera, aguardando que todas as threads concluam suas rotinas.

B. O Cozinheiro

O cozinheiro funciona sob o padrão clássico de um produtor reativo. Ele não cozinha de forma ininterrupta, na prática passa a maior parte do tempo adormecido, bloqueado pelo seu semáforo específico. Quando o caldeirão atinge zero porções, um consumidor (guerreiro ou cacique) o acorda.

Ao despertar, o cozinheiro tranca o acesso ao caldeirão utilizando o mutex correspondente e, em seguida, simula o tempo de preparo da refeição. Esse bloqueio é essencial para garantir que ninguém tente se servir de uma panela que ainda está sendo enchida. Após reabastecer as porções até o limite máximo, o cozinheiro destranca o caldeirão, avisa a thread que o acordou - e outras threads que possivelmente estavam esperando mais comida ficar pronta - de que a comida está pronta e volta a dormir, aguardando o próximo chamado.

C. Guerreiros e Caciques

As funções que definem o comportamento dos guerreiros e dos caciques compartilham exatamente a mesma lógica. No problema proposto, os dois caciques não possuem prioridade sobre os demais guerreiros, e atuam de maneira funcionalmente idêntica a uma terceira vila composta por apenas dois indivíduos. Por esse motivo, a implementação do comportamento desses três grupos (vila 1, vila 2, e caciques) é quase igual, diferindo-se apenas nas variáveis de concorrência utilizadas, e nas mensagens de monitoramento impressas no terminal.

A rotina de alimentação desses grupos é dividida em três partes. Na primeira etapa - o protocolo de entrada - o indivíduo precisa passar pela catraca geral (que garante a justiça entre os grupos, ou seja, nenhum grupo irá monopolizar os recursos) e, em seguida, registrar sua presença no interruptor do seu grupo (Vila 1, Vila 2 ou Caciques). O primeiro membro de um grupo a entrar é responsável por trancar o mutex da mesa, fazendo com que outros indivíduos da mesma facção ainda possam acessar o recurso compartilhado (mesa de jantar), mas impedindo fisicamente que qualquer facção rival se aproxime.

Na segunda etapa, a zona de alimentação, o indivíduo disputa uma vaga física nas cadeiras. Ao conseguir sentar, ele atualiza o contador de cadeiras ocupadas de forma segura (via mutexes específicos para proteger cada contador) e se dirige ao caldeirão. Se não houver comida, ele deve sinalizar (acordar) o cozinheiro e aguardar até o fim do preparo de comida. Caso contrário, ele consome uma porção, simula o tempo de refeição, atualiza o contador de lugares na mesa (indicando que levantou) e libera a vaga da cadeira.

A terceira e última etapa é o protocolo de saída. O indivíduo avisa ao seu grupo que está indo embora e, se ele for o último membro da sua facção a sair da área de alimentação, ele tem o dever de apagar a luz, ou seja, destravar a mesa principal para que os grupos rivais possam finalmente entrar.

A função descrevendo a implementação do comportamento dos guerreiros está listada em 1.

D. Algoritmos e Escolhas de Design

Os principais problemas a serem resolvidos nessa implementação foram os bloqueios eternos e garantir um acesso justo para todos - starvation e deadlocks. Para isso, foram utilizados dois algoritmos clássicos de sincronização: o interruptor (lightswitch) e a catraca (turnstile).

O interruptor, implementado com o auxílio de contadores e mutexes locais para cada grupo, garante a exclusão mútua exigida pela regra de negócios: apenas uma tribo pode comer por vez. No entanto, se uma vila muito numerosa continuasse enviando membros sem parar, a "luz" nunca se apagaria, e a vila rival morreria de fome (starvation).

Por isso a necessidade de utilizar uma catraca. O semáforo da catraca cria um gargalo obrigatório antes do interruptor. Se a Vila 1 está comendo e um membro da Vila 2 tenta entrar, ele passa pela catraca e fica bloqueado aguardando a liberação da mesa. Como ele ainda está em posse da catraca, nenhum novo membro da Vila 1 consegue passar. Isso corta o fluxo de "reforços" da vila dominante, forçando o esvaziamento natural da sala e garantindo uma alternância justa entre os grupos.

IV. CONCLUSÃO

A modelagem do problema das Vilas de Canibais Guerreiros demonstrou ser um excelente ambiente prático para o estudo de programação concorrente, pois unifica as complexidades de diversos desafios clássicos da literatura em um único cenário. A dinâmica estabelecida entre o cozinheiro e os canibais ilustra perfeitamente o problema do produtor-consumidor, no qual um recurso limitado precisa ser repostado e exaurido de maneira sincronizada. Em paralelo, a disputa pelo espaço físico da mesa introduz exigências rigorosas de controle de capacidade e exclusão mútua em grupo, remetendo fortemente à lógica do problema dos leitores e escritores, onde o acesso a uma zona crítica deve ser categoricamente isolado para uma facção por vez.

Para transpor esse modelo conceitual para um código funcional, o projeto fez uso de algumas primitivas de sincronização tradicionais. Semáforos foram aplicados como controladores de vagas físicas e sinalizadores de estado entre threads independentes, enquanto mutexes garantiram a proteção e a integridade de variáveis sensíveis, evitando corrupção de dados por condições de corrida. A administração dessas ferramentas foi guiada por padrões de design consolidados, notavelmente o interruptor (lightswitch), que permitiu o agrupamento otimizado de indivíduos de uma mesma tribo na área de alimentação, e a catraca (turnstile), que atuou como uma barreira lógica de justiça social entre os processos.

A implementação foi capaz de cumprir todos os requisitos estabelecidos para a estabilidade e funcionalidade do sistema. A arquitetura desenvolvida garantiu o respeito irrestrito às propriedades de segurança, impedindo acessos indevidos e misturas de facções rivais, ao mesmo tempo em que assegurou as propriedades de vivacidade essenciais. A combinação da catraca com as travas locais eliminou completamente o risco de impasses (deadlocks) e de inanição (starvation), forçando o revezamento ordenado dos recursos. O resultado final é um simulador que demonstra como o controle adequado do

acesso simultâneo permite que múltiplas entidades operem de forma independente, resultando em um comportamento global harmonioso e livre de colisões.

```
61 return NULL;
62 }
```

Listing 1. Função de comportamento dos guerreiros da Vila 1

REFERÊNCIAS

- [1] A. Silberschatz, P. B. Galvin, and G. Gagne, *Fundamentos de Sistemas Operacionais*, 9th ed. Rio de Janeiro: LTC, 2015.
- [2] A. S. Tanenbaum and H. Bos, *Sistemas Operacionais Modernos*, 4th ed. São Paulo: Pearson Education do Brasil, 2016.
- [3] W. Stallings, *Operating Systems: Internals and Design Principles*, 8th ed. Boston: Pearson, 2014.
- [4] E. W. Dijkstra, "Cooperating sequential processes," *Technological University, Eindhoven, The Netherlands*, 1965, artigo histórico republicado em F. Genuys (Ed.), *Programming Languages*, Academic Press, New York, 1968.

```
1 void* guerreiro_vila1(void* arg) {
2     int id = *(int*)arg;
3     while(1) {
4         sleep(2);
5         // --- Entrada: garante exclusão entre
6         grupos rivais (catraca + lightswitch) ---
7         sem_wait(&sem_catraca);
8
9         pthread_mutex_lock(&mutex_vila1);
10        vila1_cont++;
11        if (vila1_cont == 1) {
12            pthread_mutex_lock(&mutex_mesa);
13        }
14        pthread_mutex_unlock(&mutex_vila1);
15
16        sem_post(&sem_catraca);
17
18        // --- ZONA DE ALIMENTAÇÃO ---
19        sem_wait(&sem_mesa);
20
21        // Atualiza as cadeiras de forma segura ao
22        sentar
23        pthread_mutex_lock(&mutex_cont_cadeiras);
24        cadeiras_ocupadas++;
25        printf("[Vila 1 - Guerreiro %d] Sentou na
26        mesa. Cadeiras livres: %d\n", id,
27        CAPACIDADE_MESA - cadeiras_ocupadas);
28        pthread_mutex_unlock(&mutex_cont_cadeiras);
29
30        // Acessa o caldeirão para se servir
31        pthread_mutex_lock(&mutex_caldeirao);
32        while (porcoes == 0) {
33            esperando_comida++;
34            if (esperando_comida == 1) {
35                // Apenas o primeiro faminto bate na
36                porta do cozinheiro
37                printf("[Vila 1 - Guerreiro %d]
38                Caldeirão vazio! Acordando cozinheiro...\n", id
39                );
40                sem_post(&sem_cozinheiro);
41            }
42            pthread_mutex_unlock(&mutex_caldeirao);
43            sem_wait(&sem_comida_pronta);
44            pthread_mutex_lock(&mutex_caldeirao);
45        }
46        porcoes--;
47        printf("[Vila 1 - Guerreiro %d] Se serviu.
48        Por es restantes: %d\n", id, porcoes);
49        pthread_mutex_unlock(&mutex_caldeirao);
50
51        sleep(2); // Tempo comendo
52
53        // Atualiza as cadeiras de forma segura ao
54        levantar
55        pthread_mutex_lock(&mutex_cont_cadeiras);
56        cadeiras_ocupadas--;
57        printf("[Vila 1 - Guerreiro %d] Terminou de
58        comer e saiu da mesa. Cadeiras livres: %d\n", id,
59        CAPACIDADE_MESA - cadeiras_ocupadas);
60        pthread_mutex_unlock(&mutex_cont_cadeiras);
61
62        sem_post(&sem_mesa);
63
64        // --- Saída: gerência da exclusão mútua
65        (apaga lightswitch) ---
66        pthread_mutex_lock(&mutex_vila1);
67        vila1_cont--;
68        if (vila1_cont == 0) {
69            pthread_mutex_unlock(&mutex_mesa);
70        }
71        pthread_mutex_unlock(&mutex_vila1);
72    }
```